CrowSphere Qwen3-VL-8B-Instruct Agent for Orak Game Agent Challenge 2025

Team CrowSphere contact@example.com

Abstract

CrowSphere is a single Orak agent that plays 2048, Super Mario, Pokemon Red, and StarCraft II with one multimodal model dependency. We use Qwen3-VL-8B-Instruct served by a local vLLM OpenAI compatible endpoint and keep decoding parameters fixed for stable inference. Across games, we compress observations into compact evidence, ask the model to output one JSON action plan, and apply deterministic validation and formatting so only legal environment actions are executed.

System Design

Model and Serving

We use **Qwen/Qwen3-VL-8B-Instruct** as the only model dependency. The model is served locally via **vLLM** with an OpenAI compatible HTTP API. The resolved model revision is recorded in `submission/MODEL_MANIFEST.json`. Inference uses bf16 weights with no quantization and temperature 0 for deterministic decoding.

Inputs

Each step provides a text observation (`obs_str`) and optionally an image. We apply deterministic image preprocessing (short side resize and JPEG encoding) and game specific text filtering to control latency and prompt length.

Outputs

The model must output a single JSON object (ActionPlan) that encodes an action chunk. The engine validates the JSON, converts it into the environment action string, and executes at most one action per step for safety and reproducibility.

Game Specific Handling

This section summarizes how prompts and action plans are tailored per game.

2048 We parse the board from `obs_str` and choose a direction with a fixed deterministic search. The agent emits one JSON action with one direction and does not rely on text generation for this game.

Super Mario We convert `obs_str` into a compact scene evidence block that highlights Mario position, nearby hazards, and the nearest threat distance. We also generate a small menu of candidate jump profiles as jump levels in range 0 to 6 and annotate each candidate with a risk report computed from deterministic geometry checks. The model outputs one JSON action with `jump_level` and an optional candidate label. The adapter validates that the jump level respects `max_jump_level`, matches the candidate menu if a label is provided, and triggers short retries when the output is inconsistent. We execute one action per step to stay aligned with fast scene changes.

Pokemon Red Each step runs deterministic decision support that infers user interface mode, map context, and milestone progress, then produces a short candidate list. A candidate is either a key sequence or an official environment tool call such as moving to a coordinate or interacting with the next object. The prompt includes progress, a compact map summary, and the candidate list. The model selects one candidate by copying it into the JSON output. The adapter requires an exact match to the provided candidates and interrupts any queued sequence when the user interface mode changes.

StarCraft II We extract key counters from the text observation such as time, resources, supply, and unit counts and format them into a stable compact block. We then provide a small menu of complete action arrays of length `num_actions`. Every action name is constrained to `action_dict` keys so the model only selects among legal environment actions. The model returns one JSON action chunk by copying one candidate action array. The engine maintains a small cross step context based on game time to keep episode boundaries and map rotation stable in evaluation settings where map metadata can be missing.

Inference Loop and Validation

At each environment step, the agent 1) builds a prompt from the current observation and a small amount of game memory, 2) queries the model when enabled (Mario, Pokemon, StarCraft II; 2048 is deterministic by default), 3) parses the JSON ActionPlan, and 4) executes at most one environment action per step. For Super Mario, we enforce one action per step to avoid stale macro actions in fast changing scenes.

Key Configuration (Default)

Item	Value

Model	Qwen/Qwen3-VL-8B-Instruct
Serving	vLLM OpenAI compatible API on localhost port 8000
Temperature	0.0
Max output tokens	512 (Mario 256)
Image preprocessing	short side 184, JPEG quality 85
Reproducibility	HF_HOME on data disk; per call JSONL logs

Training

We do not rely on additional fine tuning in the final agent. Improvements come from prompt design, structured JSON outputs with parsing and repair, and fixed generation parameters for stable inference.

Reproducibility and Logging

We produce the following artifacts for official verification and tie break metrics.

Artifact	What it contains
`llm_calls.jsonl`	Per call records with retokenizable text, redacted images, and prompt token counts (cl100k_base).
`MODEL_DECLARATION.json`	Model name and provider plus inference call counts.
`EVALUATION_SUMMARY.json`	Overall scores and total calls and tokens.
`PER_EPISODE_BREAKDOWN.json`	Per episode scores plus per episode calls and tokens.
`MODEL_MANIFEST.json`	Model source revision and file checksums.

We provide server scripts to reproduce the full evaluation on a Linux+NVIDIA machine with vLLM.

To reproduce runs, start a local vLLM server for Qwen3-VL-8B-Instruct and run the starter-kit with uv.

Example commands 1) `vllm serve Qwen/Qwen3-VL-8B-Instruct --trust-remote-code --dtype bfloat16 --max-model-len 8192 --gpu-memory-utilization 0.90 --limit-mm-per-prompt.image 1 --limit-mm-per-prompt.video 0 --host 0.0.0.0 --port 8000 --served-model-name Qwen/Qwen3-VL-8B-Instruct` 2) `uv sync` 3) `uv run python run.py --local`

Evaluation Summary

Official REMOTE evaluation achieved full score across 12 episodes. The summary file is `submission/eval_artifacts/20260205_073454_online_309465/EVALUATION_SUMMARY.json`.